

HyRACC: A Hybrid Retrieval-Augmented Framework for More Efficient Code Completion

Chuanyi Li¹, Jiwei Shang¹, Yi Feng^{1*}, Bin Luo¹

¹ National Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China

* Corresponding author

lcy@nju.edu.cn, jiweishang@smail.nju.edu.cn, {fy,luobin}@nju.edu.cn

Abstract—Retrieval-Augmented Generation (RAG) approaches have significantly advanced code completion tasks, addressing limitations like the use of updated third-party libraries and new project dependencies. However, existing RAG methods often face challenges in balancing retrieval costs and completion accuracy. In this paper, we introduce HyRACC, a hybrid retrieval-augmented framework designed to enhance code completion efficiency. HyRACC incorporates a novel approach that uses hybrid databases at both block-level and token-level, coupled with a two-step retrieval scheme. This structure not only ensures high retrieval accuracy but also boosts response speed and reduces storage requirements. Our experimental results demonstrate that HyRACC improves code completion accuracy while optimizing latency and storage usage. Remarkably, HyRACC operates independently of model parameters, which facilitates its integration with various models and domains. This flexibility and efficiency make HyRACC particularly suitable for integration into plugins or local deployment, meeting diverse user needs for personalized code completion.

Index Terms—Domain Adaptive Code Completion, Retrieval-Augmented Generation

I. INTRODUCTION

In recent years, the rapid development of large language models (LLMs) has revolutionized the field of code completion, with various of them achieving state-of-the-art performance in general code completion tasks [1–5]. However, these models still face limitations in certain specific domains, as shown in Fig. 1. For example, large models struggle to maintain up-to-date knowledge, leading to poor performance with frequently updated third-party libraries [6]. The cloud-based operation raises concerns about code information leakage [7], and the large size of these models results in high resource overhead, etc. Therefore, in certain specific domains, users require more personalized code completion services, which also place higher demands on the efficiency of the completion models.

Small to medium-scale models coupled with RAG are seen as solutions to those problems. On one hand, these models are well-suited for local deployment. On the other hand, RAG technology enhances performance by utilizing external information, allowing for a reduction in model scale while maintaining or even improving completion performance [8]. Currently, there are two mainstream RAG methods in code completion: prompt-based RAG and logit-based RAG [9]. Prompt-based RAG constructs a block-level database, merges retrieved code blocks with incomplete code, and feeds them as

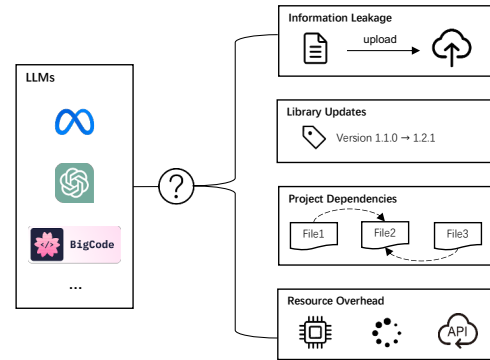


Fig. 1. Challenges faced by LLMs in specific domains of code completion, such as information leakage, library updates, project dependencies, and resource overhead.

the prompt into the model. This approach has a fast retrieval speed but is limited by the input text length and inference capabilities of the model. Logic-based RAG creates a token-level database and predicts the next token by linearly interpolating the base LM's logits with the retrieved nearest neighbor distribution. While this method supports longer contexts, it requires training and maintaining a large-scale vector database, also resulting in slower response speed. These issues hinder the advancement of customized code completion services.

In this paper, We propose HyRACC, an efficient **Hybrid Retrieval-Augmented** framework for **Code Completion**. This architecture employs innovative hybrid retrieval-augmented technology. Specifically, it first retrieves similar code snippets from a block-level database, filtering out a large amount of irrelevant information. Then, based on these similar code snippets, it constructs a small-scale token-level database and performs token retrieval, improving response speed and reducing storage space. Finally, it adaptively integrates the retrieved information with the model results based on logits for inference.

We conducted evaluations of HyRACC on several domain-specific code completion datasets. Experimental results indicate that on small LLMs, such as CodeGPT and UniXcoder, HyRACC achieved an average improvement of 11.5% in EM and 18.7% in ES. Meanwhile, it also performed well in response speed and GPU usage. Furthermore, we also achieved promising results on larger-scale models like StarCoder-1B. In

summary, the main contributions of this paper are as follows:

- We present HyRACC, a hybrid RAG framework for code completion. This framework improves code completion efficiency by constructing hybrid retrieval databases, performing hybrid retrieval, and employing adaptively inference methods. Relevant data, code, and experimental results can be found in our anonymous repository¹.
- We validate HyRACC on models of different scales and datasets from various domains. The results indicate that compared to existing methods, HyRACC shows improvements in code completion accuracy, response speed, and GPU usage.

II. RELATED WORK

A. Domain Specific Code Completion

Auto code completion is a technique that automatically provides code snippet suggestions to developers during the programming process. In addition to the significant advancements in general code completion performance driven by large models[1–5], many efforts have started to focus on domain-specific code completion tasks. For example, there are completions tailored for specific repositories [10–12] and those designed for particular scenarios [13, 14]. Some studies have gone further by introducing a domain-adaptive code completion framework that can adjust to various fields [15, 16]. Similar to these works, our model can adapt to different domains of code simply by switching databases without the need for additional fine-tuning or structural changes.

B. Information Retrieval

Retrieval is to identify and obtain relevant information given an information need. There are two primary approaches to information retrieval. Sparse retrieval methods, such as Term Frequency-Inverse Document Frequency (TF-IDF) [17] and BM25 [18], represent documents and queries using sparse vectors based on term frequencies. These methods are efficient and interpretable but struggle with issues like synonymy and polysemy. Dense retrieval methods utilize dense vector representations generated by neural network models, like BERT [19] and UniXcoder [20]. These vectors capture semantic meanings, making the system more robust in understanding semantics. However, dense retrieval methods require substantial computational resources and large amounts of annotated data.

C. RAG Foundations

1) *Prompt-based RAG*: Stemming from the idea of prompt augmentation, Prompt-based RAG integrates user queries with retrieved information and uses them as inputs for the Language Model (LM). This approach represents the mainstream paradigm for code completion. Drain et al. [21] retrieved a template of functions for function completion. RepoCoder [22] uses iterative retrieval generation to correct errors. DRACO [23] employs data flow information to assist in completion.

Repobench [24] constructs the retrieved information into a simple chain of thought to accomplish repository-level completion tasks.

2) *Logits-based RAG*: In logit-based RAG, generative models integrate retrieval information through logits during the decoding process. KNN-LMs [15] combine LM-generated probabilities with those derived from retrieval distances of similar prefixes at each decoding step. KNM-LM [16] performs logit-based RAG, combining the logits of retrieval and generation using Bayes inference.

III. APPROACH

Generally, code completion is based on the context of the code input $x = (x_1, x_2, \dots, x_n)$ and predicts a series of possible subsequent code tokens $y = (y_1, y_2, \dots, y_t)$. HyRACC effectively enhances code completion performance in specific domains by a novel RAG framework, as illustrated in Fig. 2. It mainly consists of three steps, which are detailed below.

A. Step1. Retrieve Code Snippets from Block-Level DB

To complete code in a specific domain using an external corpus D_{domain} , we first process the code files in D_{domain} by extracting function blocks, removing unnecessary blank lines, and splitting them into code snippets of less than m tokens each. These operations improve retrieval results, as noted by Karpukhin et al. [25]. Consequently, we obtain N code snippets c_i forming the block-level database: $D_{block} = \{c_1, c_2, \dots, c_N\}$.

Then, given an incomplete code x , the sparse retriever $R_S(\cdot, \cdot)$ swiftly identifies the top- M similar code snippets C_M in the D_{block} based on specific term matching metrics. The workflow of sparse retriever can be formalized as $R_S(q, D_{block}) \rightarrow C_M$. This step acts as a filter to eliminate deceptively similar tokens in irrelevant contexts, thereby narrowing down the search scope for subsequent steps. The sparse retriever we use is BM25, based on the implementation of ElasticSearch².

B. Step2. Build and Retrieve Token-Level DB

1) *Build Token-Level Database*: Inspired by k NN-LMs [15], upon the results C_M from sparse retrieval, we then construct a token-level database stored as key-value pairs. Let $f(\cdot)$ be the function that maps a context c to a fixed-length vector representation computed by the pre-trained LM, and w be the next target token of c . Then, for a sample (c_i, w_i) in C_M , we define the key-value pair (k_i, v_i) , where the key k_i is the vector representation of the context $f(c_i)$ and the value v_i is the target token w_i . Thus, the whole token-level database D_{token} can be defined as:

$$(\mathcal{K}, \mathcal{V}) = \{(f(c_i), w_i) | (c_i, w_i) \in C_M\} \quad (1)$$

The size of the D_{token} is proportional to the number of tokens in the corpus, which may lead to high computational resource

¹<https://github.com/CallMeAHuang/HyRACC>

²<https://www.elastic.co>

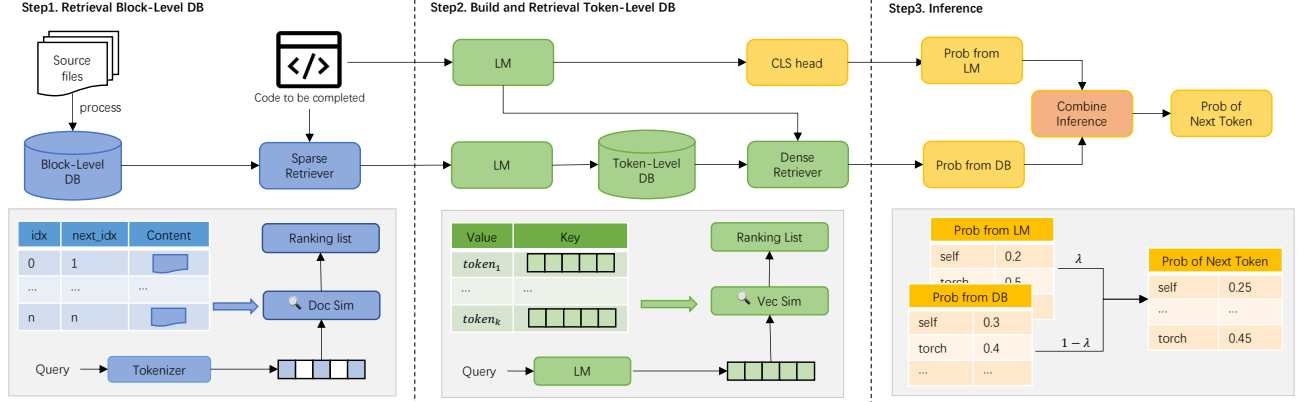


Fig. 2. The overall framework of HyRACC. It mainly consists of three steps: Step1, Retrieve code snippets from block-level database. Step 2, Build and retrieve the token-level database. Step 3, inference. The upper part of the figure depicts the main process, while the lower part shows the implementation details.

and storage space costs. However, after the snippets retrieval in Step 1, the corpus C_M is relatively small. Additionally, since most current models are derived from the transformer architecture[26], we can batch encode the corpus in parallel and then directly obtain all the keys from the KV cache, thereby optimizing latency and storage.

2) **Retrieve Tokens from Database:** With the embedded query vector $f(x)$ generated by the LM, the dense retriever $R_D(\cdot, \cdot)$ queries the database D_{token} with $f(q)$ to retrieve its k -nearest neighbors $\mathcal{N}$ base on keys similarity according to a distance function $d(\cdot, \cdot)$ (We using $L2$ distance in our experiments). The workflow of dense retriever can be formalized as $R_D(f(q), D_{token}) \rightarrow \mathcal{N}$. Then, we follow k NN-LMs [15] to generate the probability distribution of the next possible token from search results:

$$p_{DB}(y|x) \propto \sum_{(k_i, v_i) \in \mathcal{N}} \mathbb{1}_{y=v_i} \exp(-d(k_i, f(x))) \quad (2)$$

It computes a distribution over neighbors based on a softmax of their negative distances while aggregating probability mass for each vocabulary item across all its occurrences in the retrieved targets. And $\mathbb{1}$ here is an indicator function. This means that items that are not in $\mathbb{1}$ are assigned with 0 probability.

C. Step3. Inference

During inference, the next token is sampled from the mixture distribution p of the non-parametric distribution from the retrieval database $p_{DB}(y|x)$ and parametric distribution from the LM $p_{LM}(y|x)$ using a flexible hyper-parameter $\lambda \in [0, 1]$:

$$p(y|x) = \lambda \cdot p_{LM}(y|x) + (1 - \lambda) \cdot p_{DB}(y|x) \quad (3)$$

The challenge lies in adaptively determining the tuning parameter λ for different domains. The value of λ reflects the confidence of the LM. Inspired by the language modeling task [27, 28], we compute λ by evaluating the results of language

modeling during the encoding process of the input sequence:

$$\lambda = \frac{P_{LM}}{P_{LM} + P_{DB}} \quad (4)$$

Here, P_{LM} represents the proportion of tokens correctly predicted by the LM out of the total tokens in the input sequence, while P_{DB} represents the proportion of correct tokens retrieved from the database. Specifically, given an input sequence $x = (x_1, x_2, \dots, x_n)$, after LM encoding, we can access the output layer to compute the language modeling results from $\hat{x} = (\hat{x}_1, \hat{x}_2, \dots, \hat{x}_n)$. Therefore, P_{LM} can be counted as:

$$P_{LM} = \frac{1}{n} \sum_{i=1}^n \mathbb{1}(x_i = \hat{x}_i) \quad (5)$$

Where $\mathbb{1}$ is the same indicator function mentioned in III-B2. It is 1 when $x_i = \hat{x}_i$, and 0 otherwise. For P_{DB} , we can use the keys from the encoding process as queries to search for tokens in the database and compute in the same way.

IV. EXPERIMENTS SETUP

A. Task and Metrics

We conducted experiments on two code completion tasks: API completion and next-line completion. The goal of API completion is to complete API calls for specific third-party libraries, while next-line completion aims to predict the next full line of incomplete code context. We measured the accuracy of the generated code by calculating Exact Match (EM) accuracy and Levenshtein Edit Similarity (ES)[29]. Additionally, we evaluated resource overhead by assessing the response speed (tokens/s) of code generation and GPU usage (GB).

B. Datasets

Our data stem from the API-Bench[30]. This dataset includes various code scenarios in two common programming languages, Java and Python. For the API completion task, we utilized the Java Android dataset processed by Tang et al. [16] as a test set, which includes 49,000 Android API invocation

data. For the next-line completion task, we extracted 3000 lines from different Python files containing deep-learning library calls as a test set. In each of these scenarios, we employ the train set from API-Bench as the external corpus.

C. Baselines

For the base models of code completion, we selected three state-of-the-art pre-trained models. There are **CodeGPT** [31], **UniXcoder** [20] and **StarCoderBase-1B** [3]. It should be noted that both CodeGPT and UniXcoder require the same dataset preprocessing during inference, while StarCoder doesn't. Therefore, to ensure fair experimental comparisons, we employed different base models for different tasks: CodeGPT and UniXcoder for API completion, and StarCoder for line completion. Additionally, to verify the efficiency of HyRACC, we select two competitive retrieval-augmented frameworks for comparison. For prompt-based RAG, we choose **BM25** as retrieval to search relevant code snippets. For logit-based RAG, we use k **NM-LM** [16] as a representative, which saves each token in the domains code into a key-value datastore, and use k NN algorithm to retrieve similar code tokens.

D. Implementation Details

All base models are sourced from Hugging Face's relevant repositories, and we use CodeGPT-small-java-adaptedGPT2 as the representation of CodeGPT. For BM25, we use ElasticSearch to implement and retrieve the top 10 similar code blocks. These are then combined with input context to construct the prompt, following the approach used in CCEval [32]. For k NM-LM, we implement the method described by Tang et al. [16], setting k (the number of retrieved nearest neighbors) to 1024, and use faiss-gpu [33] to accelerated the k nearest neighbors search. For HyRACC, in step 1, we use BM25 to retrieve the top 10 blocks and their successors, with each block containing less than 200 tokens. In step 2, we search the top 20 tokens to construct a probability distribution. Also, to ensure fairness in the analysis of time and space complexity, all experiments are performed on a machine equipped with 2 NVIDIA 3090Ti-24GB GPUs.

V. RESULTS AND ANALYSIS

We present the experimental results in Table I and Table II. In summary, compared to the baselines, our framework shows improvements in both ES and EM metrics. Moreover, HyRACC also optimizes latency and storage compared to other retrieval approaches.

A. API Invocation Completion

We evaluated HyRACC on the Java Android API completion dataset using the CodeGPT and UniXcoder models. HyRACC achieved almost the best performance on ES and EM, even surpassing the full-scale k NM-LM, which is also based on logits. We speculate this is because HyRACC filters out deceptively similar tokens in irrelevant contexts during the sparse retrieval, thereby reducing noise. Moreover, the

average GPU usage of HyRACC is slightly lower than prompt-based RAG models represented by BM25. This is due to the quadratic increase in computational overhead with increasing prompt length in attention-based models [26]. Regarding response speed, the improvement with HyRACC is not significant as it requires real-time database generation during retrieval. However, since developers typically expect to receive completion results within 200 milliseconds [34], we consider this response speed to be acceptable.

TABLE I
RESULTS OF THE JAVA ANDROID API COMPLETION

Model	Java Android API Completion			
	ES	EM	Speed (tokens/s)	GPU (GB)
CodeGPT	43.43	4.93	323.44	2.10
+BM25	45.74	6.99	253.47	3.62
+ k NM-LM	56.42	20.17	122.97	10.83
+HyRACC	56.21	21.40	158.60	3.03
UniXcoder	48.84	3.17	345.91	2.05
+BM25	49.87	4.32	304.99	3.53
+ k NM-LM	55.84	22.52	148.73	10.05
+HyRACC	58.78	24.10	168.72	2.90

B. Line Completion

In line completion, we evaluated the performance of HyRACC using the Python DL dataset. As shown in table II, HyRACC showed improvements in EM and ES metrics, while BM25 achieved the best results. We speculate that this improvement of BM25 is due to the enhanced reasoning capabilities of large models and their support for longer contexts. In terms of response speed and GPU usage, our method outperformed several other retrieval-augmented frameworks. Additionally, as the model size and input context length increase, prompt-based RAG tends to consume more resources, which is due to the attention architecture module of the transformers [26], while HyRACC is relatively less affected.

TABLE II
RESULTS OF THE PYTHON DEEP LEARNING LINE COMPLETION

Model	Python DL Line Completion			
	ES	EM	Speed (tokens/s)	GPU (GB)
StarCoder-1B	51.60	21.26	131.16	7.10
+BM25	52.69	23.03	64.29	10.16
+ k ran s-LM	52.05	22.13	70.11	14.39
+HyRACC	52.25	22.24	84.30	8.95

VI. CONCLUSION AND FUTURE WORK

In this paper, we propose HyRACC for efficient domain-adaptive code completion. HyRACC employs innovative hybrid retrieval-augmented techniques to reduce response time and storage space. It also adaptively integrates model results and retrieval information through logits, enhancing performance in specific code domains. Notably, our framework operates without accessing model parameters, making it suitable for integration as plugins or for local deployment.

Although HyRACC has shown promising results in current experiments, there are potential limitations. Our future work plans to further explore the efficiency of HyRACC on larger-scale models and across more code domains. Additionally, we will consider introducing a memory replacement mechanism to address the issue of frequent construction of token-level databases, thereby better serving other completion tasks such as function-level completion.

ACKNOWLEDGMENTS

This work was supported by CCF-Huawei Populus Grove Fund. We want to thank the reviewers for their helpful comments.

REFERENCES

- OpenAI, Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, F. L., Almeida, D., Altenschmidt, J., Altman, S., Anadkat, S. *et al.*, “Gpt-4 technical report,” 2024. [Online]. Available: <https://arxiv.org/abs/2303.08774>
- Rozière, B., Gehring, J., Gloeckle, F., Sootla, S., Gat, I. *et al.*, “Code llama: Open foundation models for code,” 2024. [Online]. Available: <https://arxiv.org/abs/2308.12950>
- Li, R., Allal, L. B., Zi, Y., Muennighoff, N., Kocetkov, D., Mou, C., Marone, M., Akiki, C., Li, J., Chim, J. *et al.*, “StarCoder: may the source be with you!” 2023. [Online]. Available: <https://arxiv.org/abs/2305.06161>
- Allal, L. B., Li, R., Kocetkov, D., Mou, C., Akiki, C., Ferrandis, C. M., Muennighoff, N., Mishra, M., Gu, A., Dey, M. *et al.*, “Santacoder: don’t reach for the stars!” 2023. [Online]. Available: <https://arxiv.org/abs/2301.03988>
- Guo, D., Zhu, Q., Yang, D., Xie, Z., Dong, K., Zhang, W., Chen, G., Bi, X. *et al.*, “Deepseek-coder: When the large language model meets programming – the rise of code intelligence,” 2024. [Online]. Available: <https://arxiv.org/abs/2401.14196>
- Mallen, A., Asai, A., Zhong, V., Das, R., Khashabi, D., and Hajishirzi, H., “When not to trust language models: Investigating effectiveness of parametric and non-parametric memories,” 2023. [Online]. Available: <https://arxiv.org/abs/2212.10511>
- Carlini, N., Tramer, F., Wallace, E., Jagielski, M., Herbert-Voss, A., Lee, K., Roberts, A., Brown, T., Song, D., Erlingsson, U., Oprea, A., and Raffel, C., “Extracting training data from large language models,” 2021. [Online]. Available: <https://arxiv.org/abs/2012.07805>
- Izacard, G., Lewis, P., Lomeli, M., Hosseini, L., Petroni, F., Schick, T., Dwivedi-Yu, J., Joulin, A., Riedel, S., and Grave, E., “Atlas: Few-shot learning with retrieval augmented language models,” 2022. [Online]. Available: <https://arxiv.org/abs/2208.03299>
- Zhao, P., Zhang, H., Yu, Q., Wang, Z., Geng, Y., Fu, F., Yang, L., Zhang, W., Jiang, J., and Cui, B., “Retrieval-augmented generation for ai-generated content: A survey,” 2024. [Online]. Available: <https://arxiv.org/abs/2402.19473>
- Zhang, F., Chen, B., Zhang, Y., Keung, J., Liu, J., Zan, D., Mao, Y., Lou, J.-G., and Chen, W., “Repocoder: Repository-level code completion through iterative retrieval and generation,” 2023. [Online]. Available: <https://arxiv.org/abs/2303.12570>
- Phan, H. N., Phan, H. N., Nguyen, T. N., and Bui, N. D. Q., “Repohyper: Search-expand-refine on semantic graphs for repository-level code completion,” 2024. [Online]. Available: <https://arxiv.org/abs/2403.06095>
- Bogomolov, E., Zhuravlev, S., Spirin, E., and Bryksin, T., “Assessing project-level fine-tuning of ml4se models,” 2022. [Online]. Available: <https://arxiv.org/abs/2206.03333>
- Dinella, E., Ryan, G., Mytkowicz, T., and Lahiri, S. K., “Toga: a neural method for test oracle generation,” in *Proceedings of the 44th International Conference on Software Engineering*, ser. ICSE ’22. ACM, May 2022. [Online]. Available: <http://dx.doi.org/10.1145/3510003.3510141>
- Nie, P., Banerjee, R., Li, J. J., Mooney, R. J., and Gligoric, M., “Learning deep semantics for test completion,” 2023. [Online]. Available: <https://arxiv.org/abs/2302.10166>
- Khandelwal, U., Levy, O., Jurafsky, D., Zettlemoyer, L., and Lewis, M., “Generalization through memorization: Nearest neighbor language models,” 2020. [Online]. Available: <https://arxiv.org/abs/1911.00172>
- Tang, Z., Ge, J., Liu, S., Zhu, T., Xu, T., Huang, L., and Luo, B., “Domain adaptive code completion via language models and decoupled domain databases,” 2023. [Online]. Available: <https://arxiv.org/abs/2308.09313>
- Robertson, S. E. and Walker, S., “On relevance weights with little relevance information,” in *Proceedings of the 20th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, ser. SIGIR ’97. New York, NY, USA: Association for Computing Machinery, 1997, p. 16–24. [Online]. Available: <https://doi.org/10.1145/258525.258529>
- Robertson, S. and Zaragoza, H., “The probabilistic relevance framework: Bm25 and beyond,” *Found. Trends Inf. Retr.*, vol. 3, no. 4, p. 333–389, Apr. 2009. [Online]. Available: <https://doi.org/10.1561/15000000019>
- Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K., “Bert: Pre-training of deep bidirectional transformers for language understanding,” 2019. [Online]. Available: <https://arxiv.org/abs/1810.04805>
- Guo, D., Lu, S., Duan, N., Wang, Y., Zhou, M., and Yin, J., “Unixcoder: Unified cross-modal pre-training for code representation,” 2022. [Online]. Available: <https://arxiv.org/abs/2203.03850>
- Drain, D., Hu, C., Wu, C., Breslav, M., and Sundaresan, N., “Generating code with the help of retrieved template functions and stack overflow answers,” 2021. [Online]. Available: <https://arxiv.org/abs/2104.05310>
- Zhang, F., Chen, B., Zhang, Y., Keung, J., Liu, J., Zan, D., Mao, Y., Lou, J.-G., and Chen, W., “Repocoder: Repository-level code completion through iterative retrieval and generation,” 2023. [Online]. Available: <https://arxiv.org/abs/2303.12570>

- 23 Cheng, W., Wu, Y., and Hu, W., “Dataflow-guided retrieval augmentation for repository-level code completion,” 2024. [Online]. Available: <https://arxiv.org/abs/2405.19782>
- 24 Liu, T., Xu, C., and McAuley, J., “Repobench: Benchmarking repository-level code auto-completion systems,” 2023. [Online]. Available: <https://arxiv.org/abs/2306.03091>
- 25 Karpukhin, V., Oğuz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., and tau Yih, W., “Dense passage retrieval for open-domain question answering,” 2020. [Online]. Available: <https://arxiv.org/abs/2004.04906>
- 26 Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I., “Attention is all you need,” 2023. [Online]. Available: <https://arxiv.org/abs/1706.03762>
- 27 Jozefowicz, R., Vinyals, O., Schuster, M., Shazeer, N., and Wu, Y., “Exploring the limits of language modeling,” 2016. [Online]. Available: <https://arxiv.org/abs/1602.02410>
- 28 Radford, A., Narasimhan, K., Salimans, T., Sutskever, I. *et al.*, “Improving language understanding by generative pre-training,” 2018. [Online]. Available: <https://openai.com/index/language-unsupervised>
- 29 Svyatkovskiy, A., Deng, S. K., Fu, S., and Sundaresan, N., “Intellicode compose: Code generation using transformer,” 2020. [Online]. Available: <https://arxiv.org/abs/2005.08025>
- 30 Peng, Y., Li, S., Gu, W., Li, Y., Wang, W., Gao, C., and Lyu, M., “Revisiting, benchmarking and exploring api recommendation: How far are we?” 2021. [Online]. Available: <https://arxiv.org/abs/2112.12653>
- 31 Lu, S., Guo, D., Ren, S., Huang, J., Svyatkovskiy, A., Blanco, A., Clement, C., Drain, D., Jiang, D., Tang, D., Li, G., Zhou, L., Shou, L., Zhou, L., Tufano, M., Gong, M., Zhou, M., Duan, N., Sundaresan, N., Deng, S. K., Fu, S., and Liu, S., “Codexglue: A machine learning benchmark dataset for code understanding and generation,” 2021. [Online]. Available: <https://arxiv.org/abs/2102.04664>
- 32 Ding, Y., Wang, Z., Ahmad, W. U., Ding, H., Tan, M., Jain, N., Ramanathan, M. K., Nallapati, R., Bhatia, P., Roth, D., and Xiang, B., “Crosscodeeval: A diverse and multilingual benchmark for cross-file code completion,” 2023. [Online]. Available: <https://arxiv.org/abs/2310.11248>
- 33 Johnson, J., Douze, M., and Jegou, H., “Billion-scale similarity search with gpus,” *IEEE Transactions on Big Data*, p. 535–547, Jul 2021. [Online]. Available: <http://dx.doi.org/10.1109/tbdata.2019.2921572>
- 34 Wang, C., Hu, J., Gao, C., Jin, Y., Xie, T., Huang, H., Lei, Z., and Deng, Y., “Practitioners’ expectations on code completion,” 2023. [Online]. Available: <https://arxiv.org/abs/2301.03846>